Practical 3

Aim : Study and design your own Cryptocurrency.

To study the design principles behind cryptocurrencies and implement a simplified custom cryptocurrency simulation using python.

Theory:

A cryptocurrency is a digital currency secured using cryptography and maintained through a decentralized blockchain. It allows users to transfer digital assets without requiring a central issuing authority.

Core design components:

- **Wallets** – Users are represented by addresses or identifiers and maintain a coin balance.
- **Transactions** – Transactions transfer coins from one user to another. A balance check is used to prevent spending more coins than the sender owns.
- **Blocks** – Pending transactions are grouped into blocks. Each block contains a reference to the previous block's hash.
- **Mining / Proof of Work** – Miners search for a nonce that produces a SHA-256 hash satisfying a specified difficulty target.
- **Mining Reward** – New coins are awarded to the miner as an incentive for mining a block.
- **Supply Policy** – The cryptocurrency can define rules such as a fixed block reward, reward halving, or a maximum supply.

Implementation – Cryptocurrency Simulation in Python

```
class DivyCoin:
    def __init__(self):
        self.balances = {
            "Alice": 500,
            "Bob": 300,
            "Carol": 200
        }
        self.transactions = []
    def transfer(self, sender, receiver, amount):
        if sender not in self.balances:
            print("Invalid sender")
            return
```

```
        if receiver not in self.balances:
            print("Invalid receiver")
            return
        if amount <= 0:
            print("Amount must be positive")
            return
        if self.balances[sender] < amount:
            print("Insufficient balance")
            return
        self.balances[sender] -= amount
        self.balances[receiver] += amount
        self.transactions.append(
            (sender, receiver, amount)
        )
        print("Transaction successful")
coin = DivyCoin()
coin.transfer("Alice", "Bob", 50)
coin.transfer("Bob", "Carol", 25)
print("\nFinal balances:")
for name, balance in coin.balances.items():
    print(name, ":", balance, "DVC")
print("\nTransaction history:")
for transaction in coin.transactions:
    print(transaction)
```

Output:

```
PS D:\ADIT_DESKTOP\SEM - 7th\BCHAIN> python practical_3.
Transaction successful
Transaction successful

Final balances:
Alice : 450 DVC
Bob : 325 DVC
Carol : 225 DVC

Transaction history:
('Alice', 'Bob', 50)
('Bob', 'Carol', 25)
PS D:\ADIT_DESKTOP\SEM - 7th\BCHAIN>
```